# Study Report on OWASP Top 10 (2021) with Practical Examples

---

**1. Introduction**

Web applications have become an essential part of modern systems, but they are also one of the most targeted areas for cyber attacks. To address common security risks, the OWASP introduced the OWASP Top 10, which highlights the most critical web application vulnerabilities.

In this study, I explored these vulnerabilities both theoretically and practically using OWASP Juice Shop, a deliberately vulnerable application designed for security learning.

---

**2. Objective of the Study**

The main objectives of this study are:

- To understand the OWASP Top 10 vulnerabilities
- To perform practical testing using Juice Shop
- To analyze real-world attack scenarios
- To learn mitigation techniques

**3. Methodology**

For this study, I set up Juice Shop locally and used Burp Suite to intercept and analyze HTTP requests.

The process followed includes:

- Identifying input fields and endpoints
- Testing them with different payloads
- Observing responses
- Verifying whether vulnerabilities exist

---

## 4. OWASP Top 10 Vulnerabilities with Examples

---

### 4.1 Broken Access Control

Broken Access Control occurs when users can access resources beyond their permissions.

**Example (Juice Shop):**

/rest/admin/application-configuration

A normal user should not access this endpoint, but improper access control may expose sensitive data.

**Impact:**
Unauthorized access and privilege escalation

**Mitigation:**
Implement proper server-side authorization and role-based access control

---

### 4.2 Cryptographic Failures

This vulnerability occurs when sensitive data is not properly protected.

**Example (Juice Shop):**
JWT tokens stored in browser storage.

An attacker who gains access to these tokens can reuse them.

**Impact:**
Data leakage and account compromise

**Mitigation:**
Use strong encryption and secure storage mechanisms

---

### 4.3 Injection

Injection occurs when untrusted input is interpreted as part of a query.

**Example:**

/rest/products/search?q=test

**Payload:**

' OR 1=1 --

This payload is used to test SQL Injection behavior.

**Impact:**
Database compromise

**Mitigation:**
Use prepared statements and input validation

---

### 4.4 Insecure Design

This refers to flaws in application design.

**Example (Juice Shop):**
No limit on login attempts, allowing brute-force attacks.

**Impact:**
Account takeover

**Mitigation:**
Implement rate limiting and secure design practices

---

### 4.5 Security Misconfiguration

Occurs due to improper system setup.

**Example:**

/ftp

Sensitive files are accessible.

**Impact:**
Information disclosure

**Mitigation:**
Disable unnecessary services and secure configurations

---

### 4.6 Vulnerable and Outdated Components

Using outdated libraries introduces risks.

**Example:**

/package.json

Displays dependencies that may have known vulnerabilities.

**Impact:**
Exploitation of known issues

**Mitigation:**
Regularly update and monitor dependencies

---

### 4.7 Identification and Authentication Failures

Weak authentication mechanisms can be exploited.

**Example (Juice Shop):**
Weak password reset functionality.

**Impact:**
Account takeover

**Mitigation:**
Use strong passwords and multi-factor authentication

---

### 4.8 Software and Data Integrity Failures

Occurs when data integrity is not verified.

**Example (Juice Shop):**
JWT token manipulation.

Attackers may modify tokens to gain admin access.

**Impact:**
Privilege escalation

**Mitigation:**
Validate tokens and use secure signing

---

### 4.9 Security Logging and Monitoring Failures

Lack of proper logging and monitoring.

**Example:**
Multiple failed login attempts are not detected.

**Impact:**
Delayed detection of attacks

**Mitigation:**
Implement logging and monitoring systems

---

**4.10 Server-Side Request Forgery (SSRF)**

Allows attackers to make requests from the server.

**Example:**

http://localhost:3000/admin

Used in URL input fields.

**Impact:**
Internal system exposure

**Mitigation:**
Validate URLs and restrict internal access

---

**5. Tools Used**

- Burp Suite
- OWASP ZAP
- sqlmap

---

**6. Key Observations**

- Most vulnerabilities occur due to improper input validation
- Authentication and access control are common weak points
- Security misconfigurations are easy to exploit

---

**7. Conclusion**

This study provided a strong understanding of web application security. The OWASP Top 10 serves as a foundation for identifying vulnerabilities.

Using OWASP Juice Shop helped in gaining practical knowledge of how these vulnerabilities work in real-world scenarios.

---

## 8. Personal Reflection

During this study, I realized that security testing is not only about using tools but also about understanding application behavior. Even small mistakes can lead to serious vulnerabilities, which highlights the importance of secure coding and continuous testing.